

Top 20 SQL Interview Questions

Top 20 PostgreSQL Interview Questions

1. Find the highest salary

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

LinkedIn@UmangTalele

2. Find the second-highest salary ★

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

LinkedIn@UmangTalele

Using PostgreSQL **OFFSET**:

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

LinkedIn@UmangTalele

3. Find the Nth highest salary ★

For the **3rd highest salary**:

LinkedIn@UmangTalele

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

General formula:

LinkedIn@UmangTalele

N-th highest → OFFSET N-1

4. Find the top 3 highest-paid employees

LinkedIn@UmangTalele

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

5. Find employees earning more than average salary ★

LinkedIn@UmangTalele

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

6. Find the highest salary in each department ★

LinkedIn@UmangTalele

```
SELECT department,  
       MAX(salary) AS highest_salary  
FROM employees  
GROUP BY department;
```

7. Find the average salary of each department

LinkedIn@UmangTalele

```
SELECT department,  
       ROUND(AVG(salary), 2) AS average_salary  
FROM employees  
GROUP BY department;
```

PostgreSQL's `ROUND()` can be used to format the numeric result.

8. Count employees in each department

LinkedIn@UmangTalele

```
SELECT department,  
       COUNT(*) AS employee_count  
FROM employees  
GROUP BY department;
```

9. Find departments having more than 5 employees ★

LinkedIn@UmangTalele

```
SELECT department,  
       COUNT(*) AS employee_count  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 5;
```

Remember:

LinkedIn@UmangTalele

WHERE → filters individual rows
HAVING → filters groups

10. Find duplicate employee names

LinkedIn@UmangTalele

```
SELECT name,  
       COUNT(*) AS count  
FROM employees  
GROUP BY name  
HAVING COUNT(*) > 1;
```

11. Find employees whose name starts with 'A'

LinkedIn@UmangTalele

```
SELECT *  
FROM employees  
WHERE name LIKE 'A%';
```

PostgreSQL-specific bonus: ILIKE

ILIKE performs a **case-insensitive** match:

LinkedIn@UmangTalele

```
SELECT *  
FROM employees  
WHERE name ILIKE 'a%';
```

This matches **Amit**, **amit**, **AMIT**, etc.

12. Find employees from Pune or Mumbai

LinkedIn@UmangTalele

```
SELECT *  
FROM employees  
WHERE city IN ('Pune', 'Mumbai');
```

13. Find employees whose salary is between 50,000 and 80,000

LinkedIn@UmangTalele

```
SELECT *  
FROM employees  
WHERE salary BETWEEN 50000 AND 80000;
```

14. Find the highest-paid employee in each department ★

Using PostgreSQL window functions:

LinkedIn@UmangTalele

```
SELECT *  
FROM (  
    SELECT *,  
        RANK() OVER (  
            PARTITION BY department  
            ORDER BY salary DESC  
        ) AS rank  
    FROM employees  
) AS t  
WHERE rank = 1;
```

PARTITION BY department means the ranking starts again for each department.

15. Find the 2nd highest salary in each department ★

LinkedIn@UmangTalele

```
SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rank
    FROM employees
) AS t
WHERE rank = 2;
```

16. Difference between ROW_NUMBER(), RANK() and DENSE_RANK()

PostgreSQL supports all three:

LinkedIn@UmangTalele

```
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
    RANK() OVER (ORDER BY salary DESC) AS rank,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

Example when salaries contain duplicates:

LinkedIn@UmangTalele

Salary	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
80000	2	2	2
80000	3	2	2
70000	4	4	3

★ This is **very important for PostgreSQL interviews**.

17. Find employees earning more than their manager ★

This uses a **self join**:

LinkedIn@UmangTalele

```
SELECT
    e.name AS employee,
    e.salary AS employee_salary,
    m.name AS manager,
    m.salary AS manager_salary
FROM employees e
JOIN employees m
    ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

The `employees` table is joined with itself.

18. Find employees who don't have a manager

LinkedIn@UmangTalele

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

19. Find the second-highest salary using a PostgreSQL CTE ★

LinkedIn@UmangTalele

```
WITH salary_rank AS (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
    FROM employees
)
SELECT salary
FROM salary_rank
WHERE rank = 2;
```

CTEs using `WITH` are frequently asked in interviews.

20. Find the top 2 employees from each department ★

LinkedIn@UmangTalele

```
SELECT *
FROM (
    SELECT
        name,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS row_num
    FROM employees
) AS t
WHERE row_num <= 2;
```

This is a **very good interview-level PostgreSQL query**.



PostgreSQL topics you should prepare

For a PostgreSQL interview, I'd recommend learning in this order:

LinkedIn@UmangTalele

Beginner

↓

SELECT

WHERE

ORDER BY

LIMIT / OFFSET

DISTINCT

LIKE / ILIKE

IN

BETWEEN

↓

Intermediate

↓

GROUP BY

HAVING

COUNT / SUM / AVG / MIN / MAX

JOINS

Subqueries

CASE

COALESCE

↓

Advanced

↓

CTE

Window Functions

ROW_NUMBER()

RANK()

DENSE_RANK()

Self JOIN

Recursive CTE

Views

Indexes

Transactions

Stored Functions

Triggers

★ The 5 queries I'd practice first

If you're preparing for an interview soon, **master these five**:

LinkedIn@UmangTalele

```
-- 1. Second highest salary
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- 2. Highest salary per department
SELECT department, MAX(salary)
FROM employees
GROUP BY department;

-- 3. Employees above average salary
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- 4. Second highest salary per department
SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rnk
    FROM employees
) t
WHERE rnk = 2;

-- 5. Top 2 employees per department
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rn
    FROM employees
) t
WHERE rn <= 2;
```

These five cover **subqueries**, **GROUP BY**, **aggregates**, **CTE/window-function concepts**, **PARTITION BY**, **ORDER BY**, and **PostgreSQL LIMIT/OFFSET**—all very useful for interviews.